Universität Rostock



VERZWEIGUNG

Joern Ploennigs · AI4SC

Midjourney: Red or Blue Pill, ref. Leonardo da Vinci

Bedingtes Verzweigen

- Eine Verzweigung führt Code nur dann aus, wenn eine Bedingung erfüllt ist
- Bedingungen sind Ausdrücke mit Boolean-Ausgabe (x) (Operatoren: `05a_Operatoren`):
  - `True` — logische Werte
  - `a` — Variablenwerte (Bool-Äquivalent)
  - `2 < 5` — alle Vergleichsoperatoren

Ziel: KI-generierte Verzweigungen lesen, Zweige verfolgen, fehlende Fälle erkennen.

Wenn ... Dann ...

- Grundform der Verzweigung: > „Wenn die Bedingung zutrifft, dann führe folgende Statements aus”
- Oft folgt außerdem ein „sonst führe stattdessen folgendes Statement aus”
- Python kennt drei Anweisungen zur Verzweigung:
  - `if` — wenn
  - `else` — sonst
  - `elif` — kurz für else-if

IF-ANWEISUNG

- Die `if`-Anweisung startet eine Verzweigung; bedingte Blöcke müssen eingerückt werden

```
if Bedingung:
    # wird ausgeführt wenn Bedingung wahr ist
    Statement1
    Statement2
```

Ist die Bedingung `True`, werden die eingerückten Statements ausgeführt.

Beispiel: if-Anweisung

```
ausgabe_aktiv = True
a = 10
b = 13
c = (a**2 + b**2) ** 0.5  # Pythagoras

if ausgabe_aktiv:
    print("Seitenlänge:" + str(c))
    umfang = a + b + c
    print("Umfang:" + str(umfang))
```

Wahrheitswerte im Alltag

- Praktische Frage: “Ist eine Liste leer?”
- Python nutzt dafür oft direkt den Wahrheitswert eines Objekts

```
bauteile = []
if not bauteile:
    print("Die Liste ist leer")
```

De-Morgan-Regeln

- `not (A and B)` entspricht `not A or not B`
- `not (A or B)` entspricht `not A and not B`
- Hilft dabei, komplexe KI-Bedingungen umzuschreiben und zu prüfen (Wird z.B. in der Logik und beim Reasoning verwendet)

Alternativen

- `else` folgt nur auf ein `if`
- Wird ausgeführt wenn die im `if` angegebene Bedingung `False` zurückgibt
- Benötigt ebenso Einrückung des bedingten Code-Blocks

```
if Bedingung:
    Statement1
    # ...
else:
    Statement1b
    # ...
```

Beispiel: Division durch 0 (implizit)

Einfache Verzweigung, Wahrheitswert von `b`

```
a = 5
b = 0
if b:  # false bei 0, wahr für != 0
    c = a / b
else:
    print("Teilung durch 0")
    c = None
```

Beispiel: Division durch 0 (explizit)

Einfache Verzweigung, expliziter Vergleich

```
a = 5
b = 0
if b != 0:
    c = a / b
else:
    print("Teilung durch 0")
    c = None
```

Verschachtelte Verzweigungen

Durch mehrfaches Einrücken können wir Verzweigungen in Verzweigungen definieren:

```
if Condition1:
    StatementA
else:
    if Condition2:
        StatementB
    else:
        StatementC
```

Beispiel: Vorzeichen testen (Verschachtelte Verzweigung)

```
if a < 0:
    print("Die Zahl ist negativ")
else:
    if a > 0:
        print("Die Zahl ist positiv")
    else:
        print("Die Zahl ist null")
```

Mehrfachverzweigung: `elif`-Anweisung

- Anstatt einer `else`-Anweisung können wir die `elif`-Anweisung verwenden, welche wie `if` eine Bedingung hat.
- Auf eine `elif`-Anweisung können folgen:
  - Eine weitere `elif`-Anweisung
  - `else`-Anweisung — wird aufgerufen wenn keine der bisherigen Anweisungen zutraf
  - Keine Anweisung — zweite Variante vom optionalen Code

Beispiel: Vorzeichen testen (Mehrfachverzweigung)

```python
if a < 0:
    print("Die Zahl ist negativ")
elif a > 0:
    print("Die Zahl ist positiv")
else:
    print("Die Zahl ist null")
```

Viel sauberer und lesbarer als verschachtelte if-else Statements!

Branch-Lesestrategie: Beispiel

Erst Beispiel, dann Methode

```python
# KI-generiert: Wert einschätzen
if wert > 100:
    status = "hoch"
elif wert > 50:
    status = "mittel"
else:
    status = "niedrig"
```

Welche Eingabe führt zu welchem Zweig? Welche Fälle fehlen?

KI-Code lesen: Verzweigungen

Lesestrategie

Für jeden `if`-Block in KI-generiertem Code:

01 **Bedingung identifizieren** — was wird geprüft?

02 **Zweige bestimmen** — wie viele Pfade gibt es?

03 **Eingaben zuordnen** — welche Werte führen zu welchem Zweig?

04 **Fehlende Fälle prüfen** — `None`, negative Werte, leere Listen?

Viele Alternativen

In Python 3.10 wurde das Match-Statement eingeführt, was es in vielen anderen Sprachen bereits gibt. Es erlaubt eine Variable mit mehreren möglichen Werten zu vergleichen.

Traditionell (viele elif):

```
if Variable == ValueA:
    StatementA
elif Variable == ValueB:
    StatementB
else:
    StatementC
```

Neu (match-case):

```
match Variable:
    case ValueA:
        StatementA
    case ValueB:
        StatementB
    case _:
        StatementC
```

KI-generierter Code: Schwellenwert-Klassifizierer

KI-Code lesen

```python
# KI-generiert: Strukturspannung klassifizieren
def classify_stress(stress_MPa: float) -> str:
    if stress_MPa < 0:
        return "Ungültig: negativer Spannungswert"
    elif stress_MPa < 150:
        return "OK"
    elif stress_MPa < 250:
        return "Warnung"
    else:
        return "Kritisch"
```

Bevor ihr diesen Code akzeptiert: Welche Bedingung hat jeder Zweig?

Zweige verfolgen: classify_stress()

Eingabe `stress_MPa`	Bedingung erfüllt	Rückgabe
`-1`	`< 0`	`"Ungültig"`
`100`	`< 150`	`"OK"`
`200`	`< 250`	`"Warnung"`
`300`	keine oben	`"Kritisch"`

Fehlende Fälle? `None`, Strings, Grenzwerte `150` und `250` exakt?

Ein Test pro Zweig

Verifikation

```
# Minimum: ein Test pro Zweig
assert classify_stress(-1)   == "Ungültig: negativer Spannungswert"
assert classify_stress(100)  == "OK"
assert classify_stress(200)  == "Warnung"
assert classify_stress(300)  == "Kritisch"
# Grenzwerte prüfen
assert classify_stress(0)    == "OK"
assert classify_stress(150)  == "Warnung"
assert classify_stress(250)  == "Kritisch"
```

→ `08a_UnitTest` : Grenzwert-Tests systematisch

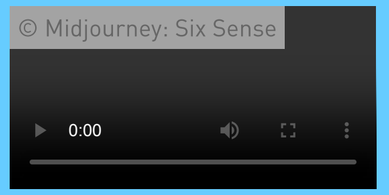
Agentic Workflow: Verzweigungen prüfen

Agentic Cycle

> **Vor dem Akzeptieren von KI-generiertem Branching-Code:**
>
> 01 Jeden Zweig manuell verfolgen
>
> 02 Fehlende Fälle identifizieren (`None`, negative Werte, Grenzwerte)
>
> 03 Einen Test pro Zweig schreiben
>
> 04 Erst dann: Code in die Lösung übernehmen

Lesson Learned

Welche Lerntypen gibt es?



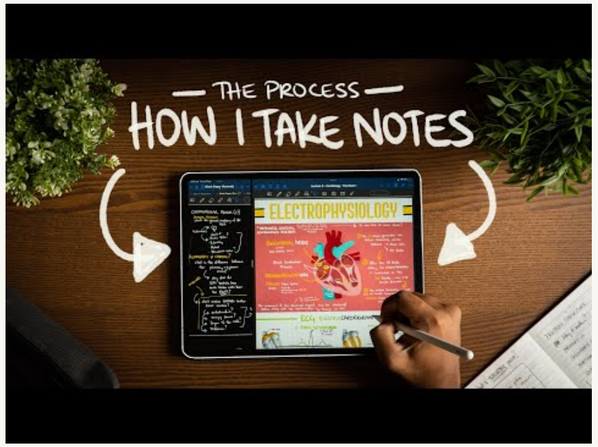
Lesson Learned - Notizen



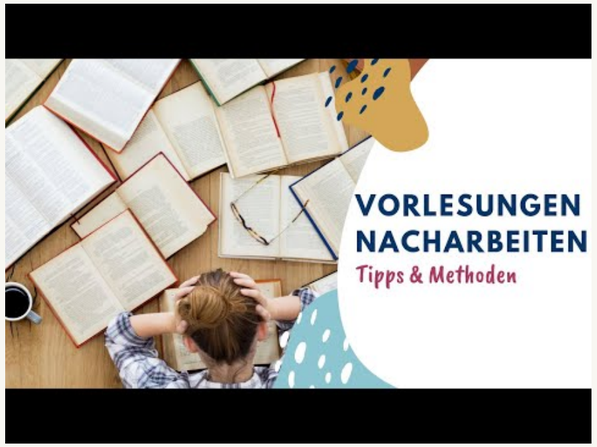
- Sie sollten sich immer Notizen machen (weil wir uns geschriebenes taktil und visuell besser merken)
- Sie sollten nicht die Folien abschreiben, sondern eher Schlüsselinformationen, Begriffe und Fragen
- Sie sollten sich die Zeit nehmen um Notizen nachzuarbeiten, Fragen zu klären und Inhalte selbst zu strukturieren

Nacharbeit

- **S**urvey (Überblick gewinnen): Blättert durch ein Thema und gewinnt einen Überblick
- **Q**uestion (Fragen): Notiert euch Fragen zu Punkten die ihr nicht versteht.
- **R**ead (Lesen): Lest eure Notizen durch; Hebt Schlüsselwörter hervor; Recherchiert zu den Fragen.
- **R**ecite (Wiedergeben): Rekapituliert jeden Abschnitt (Inhalt, Schlüsselwörter, Zusammenhänge). Strukturiert euch die Inhalte, z.B. mit Mindmaps. Wiederholt Gelerntes mehrmals (Karteikarten).
- **R**eview (Rückblick): Wiederholt gedanklich die wichtigsten Punkte und beantwortet eure Fragen.



https://youtu.be/n0ql-yeY9u0



https://youtu.be/N0NST7mwhX8

QUESTIONS